

Lab Assignment 4a
Hamming-Distance and Similarity Engine

1 Introduction

Digital systems frequently compare binary words that may differ because of noise, measurement variation, or transmission errors. The *Hamming distance* between two equal-length binary words is the number of bit positions at which they differ. For example,

$$d(10110100, 10011100) = 2.$$

In this assignment you will design an entirely combinational circuit that computes the Hamming distance between two 8-bit words and determines whether they are sufficiently similar.

Note: This assignment is simulation-based. The FPGA implementation and Viva will be part of the follow-up assignment (Lab 4b).

2 Functional Specification

For inputs $A = A_7 \dots A_0$ and $B = B_7 \dots B_0$, first define the bit-difference vector

$$X = A \oplus B.$$

The required distance is the number of 1s in X :

$$d(A, B) = \sum_{i=0}^7 X_i, \quad 0 \leq d(A, B) \leq 8.$$

The core module must have the following interface. You may add internal modules and signals.

```
module hamming_distance8 (  
    input  wire [7:0] A,  
    input  wire [7:0] B,  
    input  wire [3:0] threshold,  
    output wire [7:0] difference,  
    output wire [3:0] distance,  
    output wire      equal,  
    output wire      within_threshold  
);
```

Inputs `threshold=9,...,15` are legal. Since the distance is at most 8, `within_threshold` must be asserted for all such thresholds.

Your circuit must produce:

- an 8-bit output `difference` representing the difference vector for observation and debugging;
- a 4-bit output `distance` representing the hamming distance;
- a 1-bit output `equal` asserted exactly when the two input words are identical; and
- a 1-bit output `within_threshold` asserted exactly when `distance ≤ threshold`.

Example test case

Consider the following example to verify your understanding of the specification:

$$A = 10110100, \quad B = 10011100, \quad \text{threshold} = 3$$

The expected outputs are:

- `difference` = 00101000 (8-bit difference vector)
- `distance` = 2 (two 1s in the difference)
- `equal` = 0 (words are not identical)
- `within_threshold` = 1 (since $2 \leq 3$)

3 Design Requirements

3.1 Population-count circuit

The main design task is to count the number of 1s in an 8-bit word. Design this circuit hierarchically using smaller combinational blocks. You must explain your decomposition and the bit width of every intermediate result in your report.

You may use:

- bitwise and reduction Boolean operators;
- half-adder/full-adder modules or small adder modules constructed by you;
- a structural comparator implemented by you; and
- module instantiation, concatenation, wiring, and combinational `case/if`.

For the synthesizable population-count datapath, a flat behavioral sum such as

```
distance = X[0] + X[1] + X[2] + X[3]
          + X[4] + X[5] + X[6] + X[7];
```

is **not permitted**.

You must implement a one-bit full adder structurally from Boolean equations and construct every wider adder by instantiating full adders or smaller structural adders using your design of choice. The `+` and `-` operators are not permitted anywhere in the synthesizable design.

You must implement a 4-bit comparator and an 8-bit comparator for the final threshold comparison and the equality check of inputs respectively. Use of `==`, `≤`, `<`, `≥` and `>` is not permitted within the synthesizable design files.

The `+`, `-`, `==`, `≤`, `<`, `≥` and `>` operators may be used in the *testbench reference model* only.

3.2 Combinational implementation

The module `hamming_distance8` must contain no clock, latch, register, or inferred memory. Every output must be defined for every possible input.

Note: It is recommended to split your code into individual modules in individual Verilog design files as they could be reused in the upcoming lab assignments.

3.3 Design properties

Your implementation must satisfy:

$$d(A, A) = 0, \quad d(A, B) = d(B, A), \quad 0 \leq d(A, B) \leq 8.$$

You should be able to explain why each property follows from your design.

4 Verification Requirements

Submit a self-checking testbench. It must automatically compute the expected distance using an independent behavioral reference model and report mismatches.

Exhaustively test all $2^{16} = 65,536$ pairs of A and B and a representative set of threshold values. Your testbench must end with a clear pass/fail summary rather than relying only on visual waveform inspection.

5 Submission and Demo Instructions

Submit the following on Gradescope:

1. all Verilog design modules
2. the self-checking testbench and simulation transcript/waveforms
3. a concise report of 2–3 pages

The report should provide a comprehensive overview of each module within the design, detailing the module's inputs, outputs, and functionality. Additionally, the report must include an explanation of the testbench and what it is verifying. The analysis should also include waveform outputs from the simulation.

6 References

- Online Verilog simulator: <https://www.edaplayground.com>